Design Patterns în Java — Teorie

Adrian · Șase lecții, șase exerciții · Strategy, Observer, Factory Method, Decorator, Builder, Singleton

Lecția 0 — Ce este un design pattern (puntea de la interfețe)

De unde venim

Ai lucrat cu interfețe: un contract cu una sau două metode, semnat de clase care n-au nicio rudă comună. Codul care folosește contractul nu întreabă CE e obiectul, ci doar dacă ȘTIE să facă lucrul cerut.

Teza de atunci: **`abstract` răspunde la „ce ești”, `interface` la „ce poți face”**. Ai folosit și un contract scris de altcineva: `Comparable`. Clasa ta implementează `compareTo`, iar `Collections.sort` o sortează fără să știe nimic despre ea, doar că respectă contractul.

Ține minte propoziția asta, pentru că e singura idee din tot modulul:

> **Depinzi de contract, nu de clasă.**

Toate pattern-urile pe care le facem sunt aceeași idee, aplicată la trei întrebări diferite.

Ce e, de fapt, un pattern

Un **design pattern** nu e o bibliotecă, o clasă gata făcută sau o sintaxă nouă. E un **nume pentru o soluție care se repetă** la o problemă care se repetă. Cineva a lovit aceeași problemă de o mie de ori, a găsit forma care merge și i-a dat un nume, ca să putem vorbi despre ea în două cuvinte în loc de zece minute.

Beneficiul e dublu:

- **Nu reinventezi** — când recunoști problema, știi deja forma soluției.
- **Comunici scurt** — spui „aici e un Strategy” și toată echipa înțelege structura, fără să citească fiecare linie.

Riscul, pe care îl vom evita: pattern-ul aplicat unde nu trebuie. Un pattern rezolvă o durere concretă. **Fără durere, fără pattern** — altfel adaugi complexitate degeaba.

Firul logic al modulului — trei întrebări

Grup	Întrebarea la care răspunde	Pattern-uri
Comportament	*cine se comportă și cum?*	Strategy, Observer, State
Structură	*cum compun obiectele?*	Decorator, Adapter
Creare	*cine și cum creează obiectele?*	Factory Method, Builder, Singleton

Modulul de acum ia șase dintre ele. Începem cu **comportamentul**, pentru că e cel mai aproape de interfețe. Primul pattern — Strategy — e practic un pas mic peste ce știi deja.

Harta modulului

Lecția	Pattern	Exercițiul	Exemplul curat
1	Strategy	`ex1` — cost de livrare	`exemple/strategy` — metode de plată
2	Observer	`ex2` — starea comenzii	`exemple/observer` — canal video
3	Factory Method	`ex3` — încărcarea utilizatorilor	`exemple/factory` — forme
4	Decorator	`ex4` — notificări	`exemple/decorator` — cafea cu topping-uri
5	Builder	`ex5` — comandă cu opționale	`exemple/builder` — pizza
6	Singleton	`ex6` — numerele documentelor	`exemple/singleton` — jurnal de aplicație

Fiecare exercițiu pornește de la un cod care **merge și afișează ce trebuie**, dar e scris prost: îl refactorizezi cu pattern-ul lecției, fără să schimbi ce afișează, apoi adaugi o cerință nouă. În `exemple/` ai pentru fiecare pattern un program curat, pe **alt domeniu**; citește-l înainte de exercițiu. Fragmentele de cod de aici sunt luate din el, iar la fiecare pattern găsești „Semnul că-ți trebuie”.

Lecția 1 — Strategy

> **Un cuvânt:** comportament interschimbabil. Închizi fiecare variantă de „cum se face ceva” într-o clasă separată care semnează același contract și o schimbi când vrei, chiar la runtime.

Problema (exemplul `exemple/strategy`: metode de plată)

Un coș de cumpărături calculează taxa în funcție de metoda de plată: card, numerar. Prima variantă la care se gândește oricine:

```
public double taxa(double suma) {
    switch (metoda) {
        case CARD:
            return suma * 0.015;
        case NUMERAR:
            return 0;
        default:
            throw new IllegalStateException("Unknown payment method");
    }
}
```

Merge. Și e o capcană. Întreabă-te ce faci când apare plata prin transfer: deschizi metoda și adaugi un `case`. Probabil mai există un `switch` la fel în altă parte (afișarea numelui metodei, de exemplu) și îl deschizi și pe ăla. Fiecare `switch` uitat e un bug tăcut.

Regula încălcată se numește **open/closed**: *o clasă ar trebui să fie deschisă la extindere, dar închisă la modificare*. Adaugi comportament nou scriind cod nou, nu editând cod care deja merge.

Ideea

Fiecare variantă devine o clasă care știe DOAR de ea și semnează un contract comun. Contextul ține o strategie și o **întreabă**:

```
public interface MetodaPlata {
    String getNume();

    double taxa(double suma);
}

public class PlataCard implements MetodaPlata {

    @Override
    public String getNume() {
        return "Card";
    }

    @Override
    public double taxa(double suma) {
        return suma * 0.015;
    }
}

public class Cos {
    private final double total;
    private MetodaPlata metoda;

    public Cos(double total, MetodaPlata metoda) {
        this.total = total;
        this.metoda = metoda;
    }

    public void schimbaMetoda(MetodaPlata noua) {
```

```
        this.metoda = noua;
    }

    public double dePlata() {
        return total + metoda.taxa(total);
    }

    public String getMetoda() {
        return metoda.getNume();
    }
}
```

O metodă nouă de plată = **o clasă nouă**. Nu atingi `Cos`, nu atingi celelalte strategii.

Cele trei roluri

Rol	În exemplu	Ce face
Strategy (contractul)	`MetodaPlata`	declară operația interschimbabilă
ConcreteStrategy	`PlataCard`, `PlataNumerar`	fiecare, o singură variantă a algoritmului
Context	`Cos`	ține o strategie, deleagă spre ea, o poate schimba

Mecanismul de dedesubt

1. **Polimorfism prin contract.** `metoda.taxa(...)` execută implementarea clasei reale din spatele interfeței.
2. **Delegare.** `Cos` nu *face* calculul, îl *pasează* mai departe. Contextul e un dispecer, nu un executant.
3. **Injectare.** Strategia intră din afară (prin constructor sau `schimbaMetoda`), nu e creată înăuntru. De-aia o poți schimba la runtime.

Detaliu: `taxa` primește suma ca **parametru**, nu o ține strategia în câmp. Așa aceeași instanță `PlataCard` servește orice coș: strategia e *fără stare*, deci refolosibilă.

Când îl folosești

- Ai **mai multe variante** ale aceluiași lucru (calcul, sortare, validare, format) și vrei să comuți între ele.
- Vezi un **`switch`/lanț de `if`** pe un „tip” care se repetă prin cod.
- Vrei să **adaugi variante** fără să atingi codul existent.

Când NU îl folosești

- Ai **o singură variantă** și nu se întrevăd altele: un `if` simplu e mai onest decât trei clase.
- Variantele diferă printr-o **singură valoare**, nu printr-un algoritm: e un parametru, nu o strategie.

Capcane frecvente

- **Context cu `switch` rămas.** Dacă `Cos` tot decide cu `if` ce strategie folosește, ai dublat problema, nu ai mutat-o.
- **Strategie cu stare de comandă.** Dacă `PlataCard` ține suma în câmp, n-o mai poți refolosi între coșuri.
- **Prea multe strategii minuscule.** Dacă variantele diferă cu o linie, poate era destul un parametru.

Semnul că-ți trebuie

Un `switch` pe un tip care alege **cum** se face ceva, repetat în mai multe metode. În `ex1` îl găsești în `Comanda`, chiar de două ori. Fiecare ramură e o variantă a aceluiași lucru: o familie de strategii.

Lecția 2 — Observer

> **Un cuvânt:** un eveniment, N reacții. O sursă își schimbă starea și îi anunță pe toți cei abonați, fără să știe cine sunt sau ce fac.

Problema (exemplul `exemple/observer`: canal video)

Un canal publică un video nou. Telefonul abonaților primește o notificare, alții primesc un email. Prima variantă:

```
public class Canal {
    private final AbonatTelefon telefon = new AbonatTelefon();
    private final AbonatEmail email = new AbonatEmail();

    public void publica(String titlu) {
        telefon.notifica(titlu);
        email.notifica(titlu);
    }
}
```

`Canal` ajunge să cunoască fiecare reacție pe nume, și le creează chiar el. O reacție în plus (o notificare pe site) înseamnă să redeschizi `publica`. O clasă care ar trebui să se ocupe de *canal* știe acum de telefon, email, site.

Ideea

Inversezi dependența. Canalul ține o listă de abonați care au semnat un contract și îi anunță pe toți la fel:

```
public interface Abonat {
    void notifica(String titlu);
}

public class Canal {
    private final List<Abonat> abonati = new ArrayList<>();

    public void aboneaza(Abonat abonat) {
        abonati.add(abonat);
    }

    public void dezaboneaza(Abonat abonat) {
        abonati.remove(abonat);
    }

    public void publica(String titlu) {
        for (Abonat abonat : abonati) {
            abonat.notifica(titlu);
        }
    }
}
```

O reacție nouă = o clasă nouă care implementează `Abonat` și se abonează. `Canal` rămâne neatins.

Cele trei roluri

Rol	În exemplu	Ce face
Subject (sursa)	`Canal`	ține abonații, îi anunță pe toți când se întâmplă ceva
Observer (contractul)	`Abonat`	declară `notifica(...)`
ConcreteObserver	`AbonatTelefon`, `AbonatEmail`	fiecare reacționează în felul lui

Mecanismul de dedesubt

1. **O listă de contracte.** `List<Abonat>` ține obiecte de clase diferite, împreună, pentru că știu toate să facă un lucru.

2. **Sursa împinge.** Nu abonații întreabă „s-a schimbat ceva?”, ci sursa îi cheamă pe ei. De-aia `notifica` primește titlul ca parametru.
3. **Abonare la runtime.** Lista se umple și se golește cât rulează programul. Un abonat vede doar ce se publică după ce s-a abonat.

Strategy vs Observer: aceeași unealtă, altă direcție

	Strategy	Observer
Câți colaboratori	UNUL (strategia curentă)	MULȚI (toți abonații)
Ce face contextul	îl întreabă: „cât costă?”	îi anunță: „s-a întâmplat ceva”
Așteaptă răspuns?	DA	NU

„A întreba” vs „a anunța”: asta e distincția de fixat. Observer nu așteaptă nimic înapoi, de-aia `notifica` întoarce `void`.

Când îl folosești

- O schimbare într-un loc trebuie să declanșeze reacții în **mai multe** locuri independente.
- Vrei să adaugi sau să scoți reacții fără să atingi sursa.

Când NU îl folosești

- Ai **un singur** ascultător și va rămâne unul: un apel direct e mai clar.
- Ai nevoie de un **răspuns** de la fiecare parte (atunci nu e Observer).

Capcane frecvente

- **Sursa care cunoaște abonații concreți.** Dacă în `publica` apare `instanceof AbonatEmail`, ai ratat pattern-ul.
- **Lista modificată în timpul anunțului.** Dacă un abonat se dezabonează din interiorul lui `notifica`, bucla `for` aruncă `ConcurrentModificationException`.
- **Anunț fără schimbare reală.** Anunți doar când s-a întâmplat ceva nou, altfel abonații reacționează degeaba.

Semnul că-ți trebuie

O metodă care, la fiecare schimbare, cheamă pe rând mai multe obiecte concrete, pe care le ține în câmpuri și le creează singură. În `ex2` asta face `Comanda.schimbaStare()`. Regula: **două reacții independente la același eveniment = semnalul; o singură reacție = apel direct.**

Lecția 3 — Factory Method

> **Un cuvânt:** creare delegată. Muți `new` din codul care decide în clase care știu fiecare să nască un singur lucru.

Problema (exemplul `exemple/factory`: forme)

Ai linii de text de forma `TIP,dimensiune` (`CERC,2`, `PATRAT,3`) și pentru fiecare vrei obiectul potrivit. Prima variantă:

```
switch (campuri[0]) {
    case "CERC":
        return new Cerc(dimensiune);
    case "PATRAT":
        return new Patrat(dimensiune);
    default:
        throw new IllegalArgumentException("Unknown shape");
}
```

Merge. Și e același `switch` de la lecția 1, în alt loc: ca să adaugi `TRIUNGHI`, redeschizi o metodă care mergea. Dar de data asta Strategy **nu** te salvează, și merită înțeles de ce.

De ce polimorfismul NU rezolvă asta

Polimorfismul alege ce **metodă** rulează pe un obiect care *există deja*. Când ajungi la `new`, decizia e luată: ai scris numele clasei în cod. „Cine decide ce clasă se naște" e altă întrebare decât „cine decide cum se comportă".

> **Polimorfismul alege ce METODĂ rulează pe un obiect care există deja. Nu poate alege ce CLASĂ se construiește.**

Ideea

Dacă `new Cerc(...)` nu poate fi ales polimorfic, îl ascunzi în spatele a ceva care **poate**: un obiect a cărui singură treabă e să construiască. Fiecare fabrică are `new`-ul ei:

```
public interface FabricaForma {
    String getTip();

    Forma creeaza(double dimensiune);
}

public class FabricaCerc implements FabricaForma {

    @Override
    public String getTip() {
        return "CERC";
    }

    @Override
    public Forma creeaza(double dimensiune) {
        return new Cerc(dimensiune);
    }
}
```

Cine încarcă liniile nu mai are niciun `switch` și niciun `new` de formă. Caută fabrica după tip:

```
for (FabricaForma fabrica : fabrici) {
    if (fabrica.getTip().equals(tip)) {
        return fabrica;
    }
}
throw new IllegalArgumentException("Unknown shape: " + tip);
```

Observă ce e `if`-ul ăsta: o comparație între **două șiruri de date**, nu o verificare de tip. O formă nouă = **o clasă nouă + o fabrică nouă**, și zero linii modificate în cel care încarcă.

Rolurile

Rol	În exemplu	Ce face
Product	`Forma`	ce se construiește, văzut prin contract
ConcreteProduct	`Cerc`, `Patrat`	produsele reale
Creator	`FabricaForma`	declară metoda care naște un produs
ConcreteCreator	`FabricaCerc`, `FabricaPatrat`	fiecare, un singur `new`

Mecanismul de dedesubt

1. **`new` e o decizie luată la scriere, nu la rulare.** Singura soluție e s-o *muți* într-un obiect care poate fi ales la rulare.
2. **Indirecție.** Nu chemi constructorul, chemi pe cineva care îl cheamă pentru tine. La Strategy delegai un *calcul*, aici delegi o *naștere*.
3. **Contractul ca tip de retur.** `creeaza` întoarce `Forma`, nu `Cerc`. Dacă după fabrică urmează un cast sau un `instanceof`, ai mutat `switch`-ul, nu l-ai eliminat.

Factory Method vs Simple Factory

Numele se folosesc amestecat. Forma de mai sus (un contract de fabrică, mai multe implementări, alese la rulare) e cea pe care o vei folosi cel mai des; unii o numesc **Simple Factory**. **Factory Method** în forma din carte e mai strâmtă: o clasă de bază face o treabă mai mare și lasă o metodă abstractă prin care **subclasele decid produsul**. Ideea e aceeași: muți `new`-ul în spatele unui contract.

Când îl folosești

- Trebuie să **alegi clasa la rulare**, după o valoare (un tip dintr-un fișier, o setare, o comandă).
- Ai un `switch` care se termină în `new`.
- Vrei să adaugi un tip nou fără să atingi codul care le folosește.

Când NU îl folosești

- Ai **o singură clasă** de construit: `new` direct e mai onest.
- Problema nu e „ce clasă”, ci „cum umplu un obiect cu multe câmpuri opționale”: pentru asta există Builder (lecția 5).

Capcane frecvente

- **Cast după fabrică.** `(Cerc) fabrica.creeaza(...)` în client = problema mutată.
- **`switch` mutat în fabrică.** O singură clasă cu `switch` înăuntru tot se redeschide la fiecare tip nou. Câte tipuri, atâtea fabrici.
- **`new` uitat în client.** Caută cuvântul `new` în cel care încarcă. Dacă apare un produs concret, pattern-ul nu e complet.

Semnul că-ți trebuie

Un `switch` pe o valoare citită la rulare, în care fiecare ramură se termină în `new`. În `ex3` îl găsești în `IncarcatorUtilizatori`.

Lecția 4 — Decorator

> **Un cuvânt:** comportament adăugat prin împachetare. O clasă care semnează contractul **și ține înăuntru un alt obiect cu același contract**, îl cheamă și îi ajustează intrarea sau rezultatul. Se pot stivui.

Problema (exemplul `exemple/decorator`: cafea cu topping-uri)

O cafenea vinde cafea simplă, cu lapte, cu caramel, cu ambele. Prima variantă e o clasă pentru fiecare combinație:

```
Cafea, CafeaCuLapte, CafeaCuCaramel, CafeaCuLapteSiCaramel,
CafeaCuLapteDublu, ...
```

Fiecare topping nou dublează numărul de clase, iar combinațiile copiază codul unele de la altele.

Ideea

Fiecare adăugire devine **o clasă care semnează același contract și ține înăuntru un obiect cu același contract**. Face partea ei, apoi dă mai departe:

```
public interface Bautura {
    String descriere();

    double pret();
}

public class CuLapte implements Bautura {
    private final Bautura interior;

    public CuLapte(Bautura interior) {
        this.interior = interior;
    }

    @Override
    public String descriere() {
        return interior.descriere() + " + lapte";
    }

    @Override
    public double pret() {
        return interior.pret() + 2;
    }
}
```

Combinațiile nu mai sunt clase, sunt **obiecte puse unul în altul**, la rulare:

```
Bautura completa = new CuCaramel(new CuLapte(new Cafea()));
System.out.println(completa.descriere() + ": " + completa.pret());
```

Rezultatul: `Cafea + lapte + caramel: 13.0`.

Cele trei roluri

Rol	În exemplu	Ce face
Component (contractul)	`Bautura`	operația pe care o semnează și cel decorat, și decoratorul
ConcreteComponent	`Cafea`	face treaba de bază, nu împachetează pe nimeni
Decorator	`CuLapte`, `CuCaramel`	semnează contractul, ține o `Bautura`, adaugă ceva și deleagă

De ce merită un nume separat de Strategy

	Strategy	Decorator

Ce ține clasa	contextul ține **strategia**	decoratorul ține **un obiect de același tip cu el**
Câte niveluri	unul	oricâte, stivuite
Cine îl vede	contextul știe că are o strategie	apelantul **nu știe** că e împachetat

Mecanismul de dedesubt

1. **Aceeași interfață în două roluri.** Decoratorul e în același timp *implementare* a contractului și *client* al lui. De-aia poate sta oriunde stă un obiect obișnuit, inclusiv în alt decorator.
2. **Compunere la runtime, nu moștenire.** Trei decoratori dau opt combinații fără nicio clasă în plus.
3. **Ordinea contează.** Dacă decoratorul lucrează **înainte** să delege, citești lanțul din afară spre înăuntru; dacă lucrează **după** (ca `CuLapte`, care adaugă la prețul întors), citești dinăuntru spre afară.
4. **Descrierea se compune.** Un decorator nu-și întoarce o descriere fixă, ci o construiește din cea dinăuntru: `interior.descriere() + " + lapte"`. Așa vezi tot lanțul.

Când îl folosești

- Vrei să **adaugi** ceva peste un comportament existent, fără să atingi clasa care îl face.
- Ai nevoie de **combinații** de adăugiri, nu de o listă fixă.
- Adăugirea e transversală: jurnalizare, validare, limitare, cache.

Când NU îl folosești

- Ai **o singură** adăugire și nu se întrevăd altele: pune-o în clasa de bază.
- Adăugirea trebuie să știe **ce anume** decorează (`instanceof Cafea`): atunci nu e decorator.
- Nu există un contract comun. Să inventezi o interfață doar ca să ai ce decora e pattern forțat.
- Vrei ca mai mulți să reacționeze la un eveniment: aia e Observer.

Capcane frecvente

- **Decorator care nu deleagă.** Dacă nu apare niciun apel către `interior`, ai scris o clasă obișnuită cu un câmp nefolosit.
- **Descriere fixă.** `return "Cu lapte";` ascunde tot ce e dedesubt. Compune-o.
- **Ordine presupusă.** Dacă rezultatul depinde de ordine, rulează ambele ordini, nu doar pe cea care-ți iese bine.

Semnul că-ți trebuie

Clase cu nume ca `XCuYSiZ`, care repetă codul unei clase de bază plus ceva, sau un lanț de steaguri `boolean` care pornesc și opresc adăugiri. În `ex4` ai șase clase de notificări, iar fiecare combinație nouă cere încă una. Fără un contract comun, fără Decorator.

Lecția 5 — Builder

> **Un cuvânt:** construcție pas cu pas. Un obiect cu multe câmpuri, dintre care unele opționale, se asamblează prin apeluri cu nume, iar verificările se fac o singură dată, la final.

Problema (exemplul `exemple/builder`: pizza)

O pizza are o dimensiune obligatorie și câteva opționale: sos, extra brânză, ciuperci. Prima variantă e un constructor cu toate câmpurile:

```
public Pizza(String dimensiune, String sos,
             boolean extraBranza, boolean ciuperci) { ... }

Pizza a = new Pizza("medie", null, false, false);
Pizza b = new Pizza("mare", "rosii", true, true);
```

Merge. Și e greu de citit. Ce înseamnă `null, false, false`? Trebuie să numeri parametrii și să te uiți în clasă. Dacă inversezi două argumente de același tip, compilatorul nu spune nimic. Și cu fiecare opțional nou, constructorul se lungește (constructorul **telescopic**) sau apar constructori noi, iar toate apelurile existente trebuie rescrise.

Alternativa la îndemână, un constructor gol și setteri, lasă obiectul pe jumătate construit și modificabil de oricine: nu mai ai nicio garanție că o `Pizza` e validă.

Ideea

Mută construcția într-un obiect ajutător, `Builder`, care primește valorile una câte una, **cu nume**, și produce obiectul abia la final. Constructorul clasei devine `private`: singurul drum spre o `Pizza` trece prin builder.

```
public class Pizza {
    private final String dimensiune;
    private final String sos;
    private final boolean extraBranza;
    private final boolean ciuperci;

    private Pizza(Builder builder) {
        this.dimensiune = builder.dimensiune;
        this.sos = builder.sos;
        this.extraBranza = builder.extraBranza;
        this.ciuperci = builder.ciuperci;
    }

    public String descriere() {
        String text = "Pizza " + dimensiune;
        if (sos != null) {
            text += " | sos: " + sos;
        }
        if (extraBranza) {
            text += " | extra branza";
        }
        if (ciuperci) {
            text += " | ciuperci";
        }
        return text;
    }

    public static class Builder {
        private String dimensiune;
        private String sos;
        private boolean extraBranza;
        private boolean ciuperci;
```

```
        public Builder dimensiune(String dimensiune) {
            this.dimensiune = dimensiune;
            return this;
        }

        public Builder sos(String sos) {
            this.sos = sos;
            return this;
        }

        public Builder extraBranza() {
            this.extraBranza = true;
            return this;
        }

        public Builder ciuperci() {
            this.ciuperci = true;
            return this;
        }

        public Pizza build() {
            if (dimensiune == null || dimensiune.isBlank()) {
                throw new IllegalArgumentException("Size is required");
            }
            if (extraBranza && dimensiune.equals("mica")) {
                throw new IllegalArgumentException(
                        "Small pizza cannot have extra cheese");
            }
            return new Pizza(this);
        }
    }
}
```

Folosirea:

```
Pizza completa = new Pizza.Builder()
        .dimensiune("mare")
        .sos("rosii")
        .extraBranza()
        .ciuperci()
        .build();
```

Cele trei roluri

Rol	În exemplu	Ce face
Product	`Pizza`	obiectul final: câmpuri `final`, fără setteri, deci imutabil
Builder	`Pizza.Builder`	ține valorile în curs de adunare; fiecare setter întoarce `this`; `build()` verifică și produce
Client	`Main`	înșiră apelurile și încheie cu `build()`

Mecanismul de dedesubt

1. **`return this`.** Un setter întoarce chiar builderul pe care a fost chemat, deci poți lega apelurile unul după altul.
2. **Clasă internă statică.** `Builder` trăiește în `Pizza`, deci vede constructorul `private`. Fiind `static`, se creează fără o `Pizza` existentă: `new Pizza.Builder()`.
3. **Validarea într-un singur loc.** Regulile pe o **combinație** de câmpuri („mică nu poate avea extra brânză”) se pot verifica doar când toate valorile sunt la dispoziție: în `build()`. Dacă trece, obiectul născut e valid pe toată viața lui.
4. **Imutabilitate.** După `build()` obiectul nu mai are cum să se schimbe: câmpuri `final`, fără setteri.

Builder vs Factory Method

	Factory Method (lecția 3)	Builder (lecția 5)
Întrebarea	**ce clasă** construiesc?	**cum asamblez** un obiect cu multe câmpuri?
Cum alegi	după o valoare citită la rulare	clasa e una singură, se alege doar ce pui în ea
Rezultatul	un produs dintr-o familie	aceeași clasă, cu combinații diferite de câmpuri

Când îl folosești

- Obiectul are **multe câmpuri**, iar o parte sunt opționale.
- Ai (sau ai ajunge să ai) apeluri cu `null` și `false` pasate la întâmplare.
- Există **reguli pe combinații** de câmpuri și vrei ca obiectul să nu existe niciodată într-o stare nevalidă.

Când NU îl folosești

- Clasa are **două-trei câmpuri**, toate obligatorii: un constructor obișnuit e mai clar.
- Câmpurile se schimbă pe parcursul vieții obiectului: obiectul nu e imutabil și Builder nu-ți aduce nimic.
- Problema e „ce clasă”, nu „cum o umplu”: aia e Factory Method.

Capcane frecvente

- **Uiți `build()`.** Lanțul compilează și rulează, dar rămâi cu un builder, nu cu o `Pizza`.
- **Validare doar în setteri.** Regula „mică fără extra brânză” nu poate fi verificată în `extraBranza()`, pentru că dimensiunea poate veni după. Se verifică în `build()`.
- **Setteri „ca să fie la îndemână”.** Un singur `setSos` în `Pizza` strică imutabilitatea și validarea.
- **Builder pentru orice.** Dacă clasa are un nume și un id, builderul e mai mult cod decât problema.

Semnul că-ți trebuie

Te uiți la un apel de constructor și nu poți spune fără să deschizi clasa ce înseamnă fiecare argument, iar printre ele sunt `null` sau `false`. În `ex5` ai un astfel de constructor cu 6 parametri.

Lecția 6 — Singleton

> **Un cuvânt:** o singură instanță, cu un punct de acces. O clasă care nu se poate construi decât o dată, iar toată lumea ajunge la aceeași instanță printr-o metodă statică.

Problema (exemplul `exemple/singleton`: jurnal de aplicație)

O aplicație are un jurnal în care scriu mai multe clase: autentificarea, backup-ul, plățile. Jurnalul numără liniile și are un singur loc unde ajung. Prima variantă: fiecare clasă își face propriul jurnal.

```
public class Backup {
    private final Jurnal jurnal = new Jurnal();
}

public class Autentificare {
    private final Jurnal jurnal = new Jurnal();
}
```

Merge, dar sunt două jurnale, fiecare cu numărătoarea lui: `[LOG 1]` din autentificare, `[LOG 1]` din backup. A doua variantă: creezi jurnalul o dată și îl pasezi prin constructori, prin toate clasele, chiar și prin cele care doar îl duc mai departe. Merge, până când cineva scrie un `new Jurnal()` într-o clasă nouă și seria se rupe, fără nicio eroare.

Ideea

Dacă ceva trebuie să existe **o singură dată** pentru toată aplicația, clasa însăși garantează asta. Constructorul devine `private` (nimeni nu mai poate scrie `new Jurnal()`), iar instanța unică e ținută de clasă și oferită printr-o metodă statică:

```
public class Jurnal {
    private static final Jurnal INSTANTA = new Jurnal();

    private final List<String> linii = new ArrayList<>();

    private Jurnal() {
    }

    public static Jurnal getInstance() {
        return INSTANTA;
    }

    public void scrie(String mesaj) {
        linii.add(mesaj);
        System.out.println("[LOG " + linii.size() + "] " + mesaj);
    }

    public int numarLinii() {
        return linii.size();
    }
}
```

Cine vrea jurnalul îl cere, nu îl construiește și nu îl primește:

```
public class Autentificare {

    public void login(String utilizator) {
        Jurnal.getInstance().scrie("Login: " + utilizator);
    }
}
```

Mesajele vin din clase diferite, dar seria e una singură:

```
Autentificare autentificare = new Autentificare();
Backup backup = new Backup();
autentificare.login("ana");
backup.ruleaza();
autentificare.login("ion");
```

```
[LOG 1] Login: ana
[LOG 2] Backup pornit
[LOG 3] Login: ion
```

Cele trei elemente

Element	În exemplu	Ce face
Constructor `private`	`private Jurnal()`	nu mai poți scrie `new Jurnal()` în afara clasei
Câmp `static final`	`INSTANTA`	ține unica instanță, creată o singură dată la încărcarea clasei
Metodă `static`	`getInstance()`	punctul de acces: întoarce mereu aceeași instanță

Mecanismul de dedesubt

1. **Un câmp `static` aparține clasei, nu unui obiect.** Există o singură copie, indiferent câte obiecte ar exista. De-aia poate ține „singura instanță”.
2. **Inițializarea statică rulează o singură dată.** JVM-ul creează `INSTANTA` când încarcă clasa prima oară, o singură dată, chiar dacă mai multe fire cer clasa deodată. Varianta simplă din exemplu este deci sigură și între fire.
3. **Constructorul `private` închide celelalte drumuri.** Singurul `new Jurnal()` din tot programul e cel din câmpul static.

Pe internet vei vedea și variante cu `enum` sau cu „double-checked locking”. Sunt optimizări și plasturi pentru cazuri speciale (creare întârziată, serializare); pentru început ți-o ajunge forma de mai sus.

Când îl folosești

- Ceva trebuie să existe **o singură dată**, ca altfel datele s-ar împărți greșit: o serie de numere (facturi, comenzi), un jurnal, un registru comun.
- Aceeași instanță e cerută din **multe locuri** și plimbarea ei prin constructori ar încărca clase care n-au treabă cu ea.

Când NU îl folosești

- Vrei doar „să ai la îndemână” un obiect: o instanță pasată prin constructor e mai clară.
- Obiectul ține date care diferă de la un utilizator la altul sau de la o cerere la alta (coșul, utilizatorul curent): acolo o instanță unică amestecă datele.
- Ai nevoie să schimbi obiectul în teste sau să-l înlocuiești cu altul: un Singleton nu se lasă înlocuit.

Capcane frecvente

- **Stare globală.** Orice clasă poate cere instanța și o poate schimba. Dacă un Singleton ține o stare care se modifică, orice clasă poate strica ce a făcut alta, fără ca asta să se vadă din semnături. Păstrează-i puțină stare și metode clare.
- **Greu de testat.** Dependența nu apare în constructor, deci un test nu o poate înlocui. Mai rău: instanța rămâne cu starea lăsată de testul anterior, iar rezultatul unui test depinde de ordinea în care rulează. În `ex6`, după un test care a luat numerele 1001–1004, un al doilea test ar începe de la 1005.
- **Dependență ascunsă.** Nu vezi din constructorul lui `Comanda` că ea ia numere din generator; trebuie să-i citești corpul. E prețul pentru că nu o mai plimbi prin parametri.

Semnul că-ți trebuie

O singură instanță care **trebuie** să fie comună (altfel seria, jurnalul sau registrul se rup) și pe care o vezi pasată prin constructori doar ca să ajungă în adânc. În `ex6` asta face `GeneratorNumere`: trece prin `Comanda` și prin `Retur`, iar un singur `new GeneratorNumere()` în plus ar da numere duplicate.

Rămase în afara acestui modul: State, Adapter.